In this section you'll learn about CSRF attacks, how attackers can use them to impersonate a user on your site, and how to protect against them using antiforgery tokens. Razor Pages protects you from these attacks by default, but you can disable these verifications, so it's important to understand the implications of doing so.

CSRF attacks can be a problem for websites or APIs that use cookies for authentication. A CSRF attack involves a malicious website making an authenticated request to your API on behalf of the user, without the user's initiating the request. In this section we'll explore how these attacks work and how you can mitigate them with antiforgery tokens.

The canonical example of this attack is a bank transfer/withdrawal. Imagine you have a banking application that stores authentication tokens in a cookie, as is common (especially in traditional server-side rendered applications). Browsers automatically send the cookies associated with a domain with every request so the app knows whether a user is authenticated.

Now imagine your application has a page that lets a user transfer funds from their account to another account using a `POST` request to the `Balance` Razor Page. You have to be logged in to access the form (you've protected the Razor Page with the `[Authorize]` attribute or global authorization requirements), but otherwise you post a form that says how much you want to transfer and where you want to transfer it. Seems simple enough?

Suppose that a user visits your site, logs in, and performs a transaction. Then they visit a second website that the attacker has control of. The attacker has embedded a form in their website that performs a `POST` to your bank's website, identical to the transfer-funds form on your banking website. This form does something malicious, such as transfer all the user's funds to the attacker, as shown in figure 29.4. Browsers automatically send the cookies